FINAL PROJECT REPORT

Smart Contract Applications

Project F4 - Document Timestamping Notary

Student: Ahmed

Date: 14 May 2026

Blockchain & Smart Contracts Course

Executive Summary

Table of Contents

This report presents the design, implementation, and validation of a decentralized document timestamping system using Ethereum smart contracts. The solution records only document hashes on-chain to provide immutable proof-of-existence while preserving document privacy.

The project includes a Solidity contract, automated Hardhat tests, a browser-based frontend connected through MetaMask, and live demonstration evidence from deployment and transaction execution.

1. Problem Statement and Motivation

Traditional timestamping relies on centralized notary services or institutional databases. These systems can introduce trust, availability, and cost constraints. In academic, legal, and commercial scenarios, users need a verifiable and tamper-resistant proof that a digital artifact existed at a specific time.

Project F4 addresses this by leveraging blockchain immutability. Instead of uploading files, the system computes a SHA-256 digest locally and notarizes the digest on-chain. Any party can later recompute the hash and verify ownership and timestamp.

2. Technical Architecture

2.1 Smart Contract Layer

- Language: Solidity ^0.8.19
- Core contract: DocumentNotary
- Storage model: mapping(bytes32 => Document)
- Security controls: valid hash guard and duplicate prevention
- Auditability: event emission on notarization and verification

2.2 Frontend Layer

- Stack: HTML/CSS/JavaScript + ethers.js v6
- Wallet integration: MetaMask via window.ethereum
- Local hashing: Web Crypto API (SHA-256)
- Network target: Hardhat local node (Chain ID 31337)

3. Smart Contract Design Walkthrough

3.1 Data Model

The Document struct stores owner address, block timestamp, free-text description, and an exists flag. The exists flag is mandatory because uninitialized mappings return zero values.

3.2 Main Functions

- notarizeDocument(bytes32, string): writes new proof and emits DocumentNotarized.
- verifyDocument(bytes32): emits DocumentVerified and returns proof details.
- getDocumentInfo(bytes32): read-only accessor used by frontend.
- getDocumentsByOwner(address): returns all hashes registered by an address.

3.3 Security and Correctness

- Rejects empty hash values (bytes32(0)).
- Rejects duplicate notarization attempts for the same hash.
- Stores only hash metadata to avoid leaking document content.

4. Implementation and Execution Procedure

The complete run sequence used in this project is listed below:

- cd "/home/ahmed/Blockchain Final Project"
- npm install
- npm run compile
- npm test
- npm run node
- npm run deploy:local
- cd frontend && python3 -m http.server 5500

The frontend was accessed at http://127.0.0.1:5500 and connected through MetaMask.

5. Test Strategy and Validation Results

Automated tests were executed using Hardhat. The suite validates ownership assignment, successful notarization, duplicate blocking, zero-hash rejection, unknown hash verification behavior, and owner-specific document retrieval.

- Result: 6 passing tests.
- Compilation status: successful.
- Local deployment status: successful.

6. Evidence Screenshots

All screenshots below are captured from the actual implementation session.

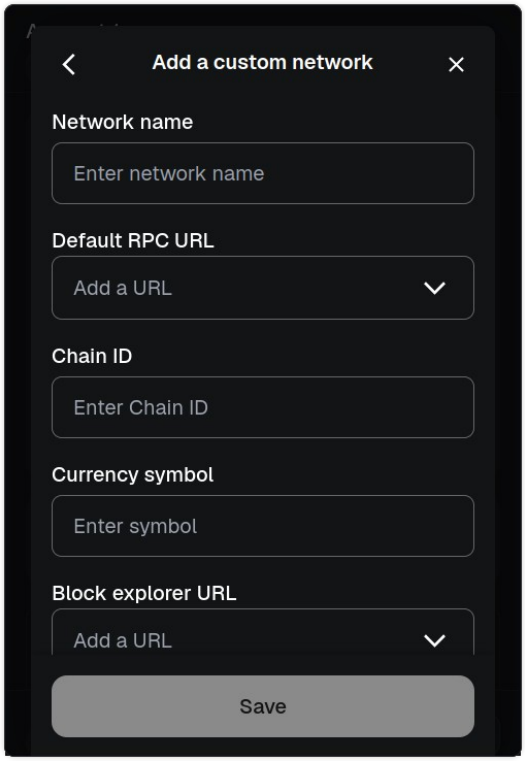

Figure 1. MetaMask custom network configuration for Hardhat Local.

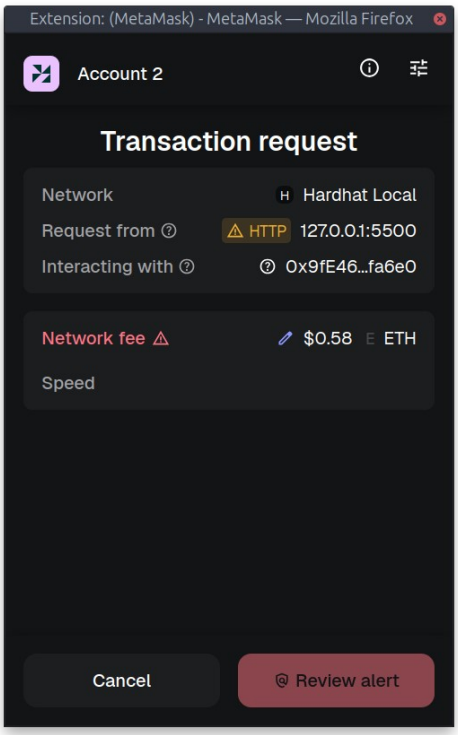

Figure 2. Transaction confirmation dialog on Hardhat Local network.

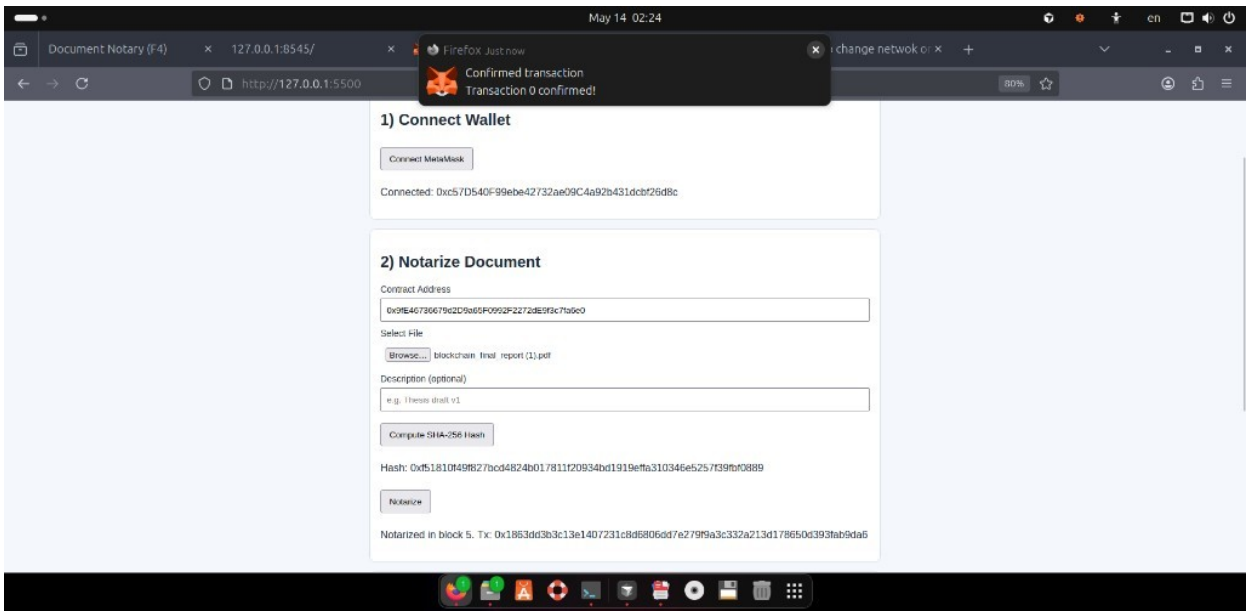

Figure 3. Notarization success message with block and transaction hash.

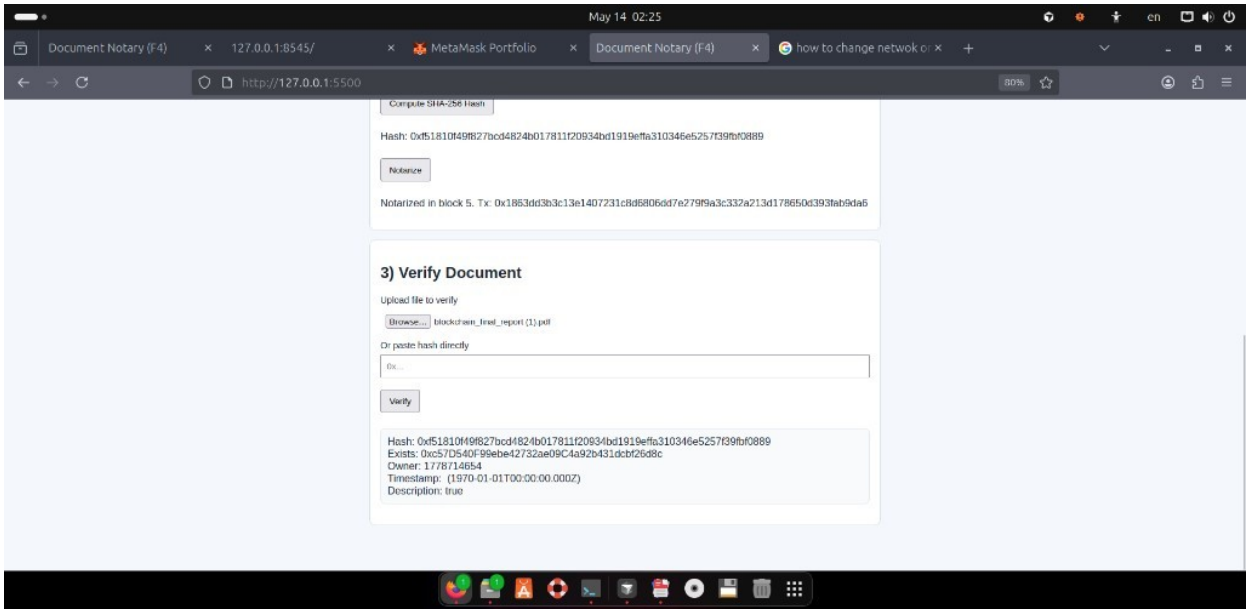

Figure 4. Verification interface during live test execution.

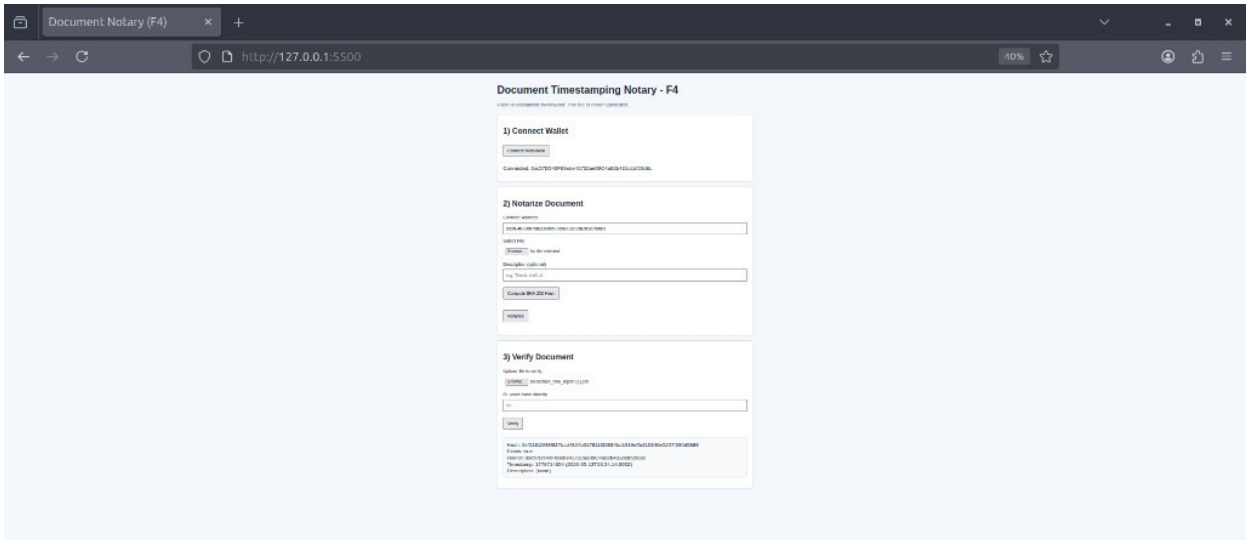

Figure 5. Final verification screen after frontend mapping fix.

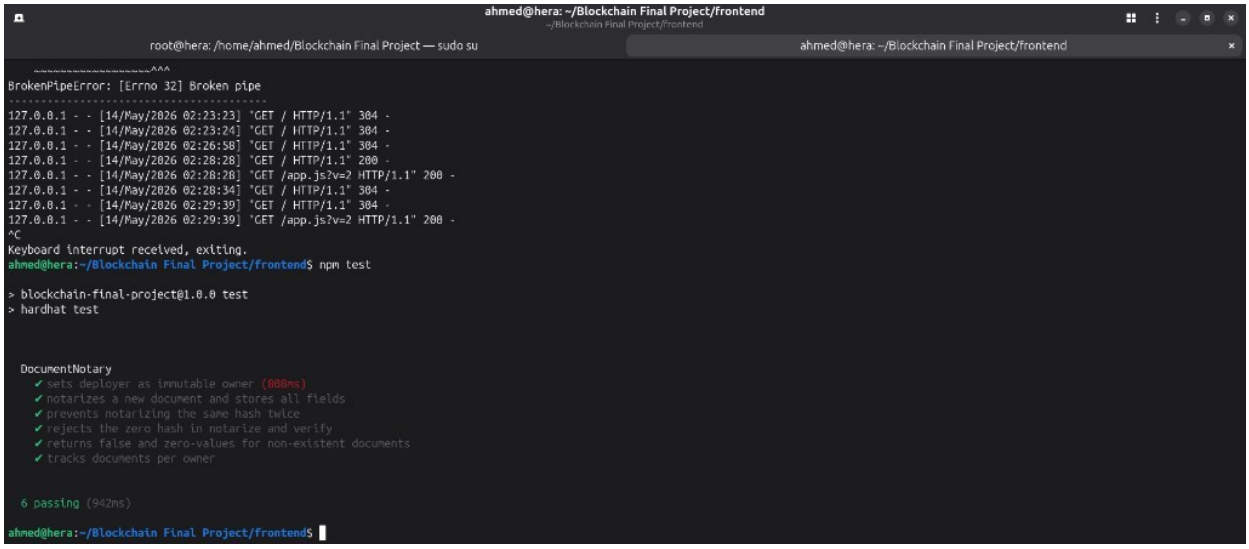

Figure 6. Terminal evidence of successful Hardhat test suite.

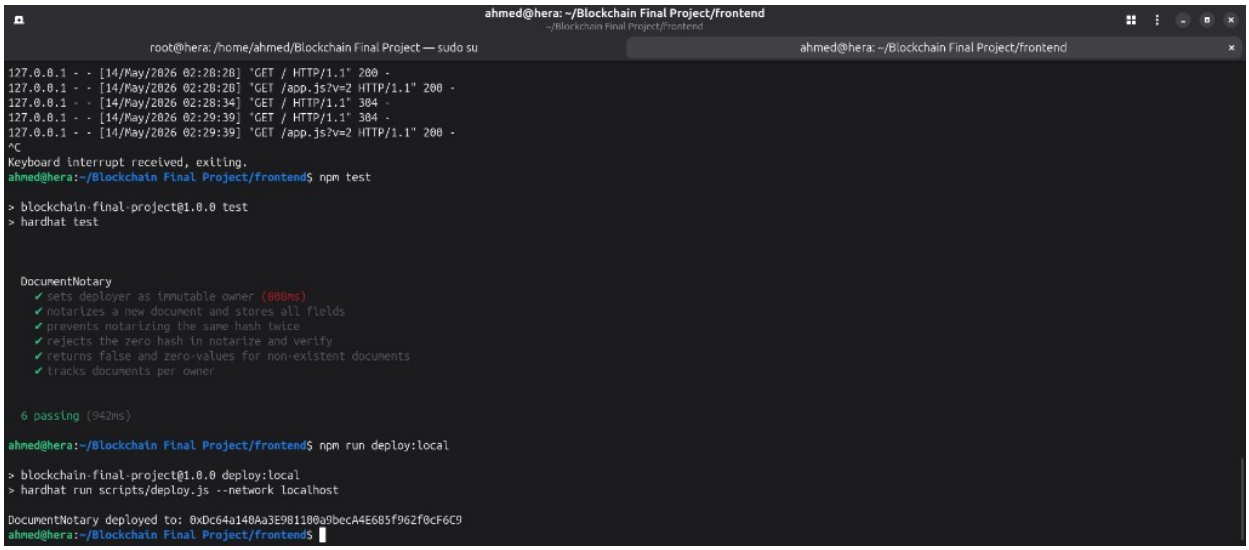

Figure 7. Terminal evidence of successful local contract deployment.

7. Challenges and Improvements

- Resolved environment issues related to Node and Hardhat compatibility.
- Handled intermittent package-network failures during setup.
- Corrected frontend tuple mapping bug to display verification fields properly.
- Added cache-busting on frontend script to avoid stale browser assets.
- Funded local MetaMask account for smooth local transaction execution.

8. Conclusion

The project goals for F4 were achieved successfully. The implemented dApp provides a reliable and privacy-preserving proof-of-existence workflow with complete smart contract functionality, automated test validation, and user-facing interaction through MetaMask.

This implementation can be extended to production-style environments by integrating testnet deployment, richer access control policies, and formal static-analysis reports.